

# Technical Guide: Building a Serverless RAG System on AWS with FAISS and Bedrock

[joepareti54@gmail.com](mailto:joepareti54@gmail.com)

11. Juni 2026

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>Introduction and Purpose</b>                  | <b>3</b>  |
| Background: the local Llama 2 prototype          | 3         |
| <b>Architecture Overview</b>                     | <b>4</b>  |
| <b>Part I: All Required Files</b>                | <b>4</b>  |
| Project Structure                                | 5         |
| Root Files                                       | 5         |
| app.py                                           | 5         |
| cdk.json                                         | 6         |
| requirements.txt                                 | 6         |
| requirements-dev.txt                             | 7         |
| .env.example                                     | 7         |
| Config Directory                                 | 7         |
| config/config.py                                 | 7         |
| Stacks Directory                                 | 9         |
| stacks/__init__.py                               | 9         |
| stacks/rag_stack.py                              | 9         |
| Lambda Functions                                 | 12        |
| lambdas/document_processor/document_processor.py | 12        |
| lambdas/document_processor/requirements.txt      | 15        |
| lambdas/index_manager/index_manager.py           | 15        |
| lambdas/index_manager/requirements.txt           | 17        |
| lambdas/query_handler/query_handler.py           | 17        |
| lambdas/query_handler/requirements.txt           | 20        |
| Layers Directory                                 | 20        |
| layers/dependencies/requirements.txt             | 20        |
| Scripts Directory                                | 20        |
| scripts/upload_documents_fixed.py                | 21        |
| Test the script                                  | 28        |
| <b>Part II: Deployment Guide</b>                 | <b>28</b> |
| Prerequisites on Ubuntu PC                       | 29        |
| Step 1: Install Required Software                | 29        |
| Step 2: Install AWS CDK                          | 29        |
| Step 3: Configure AWS Credentials                | 29        |
| Prerequisites in AWS Console                     | 30        |

|                                                    |           |
|----------------------------------------------------|-----------|
| Step 4: Enable Bedrock Model Access                | 30        |
| Create the Project                                 | 30        |
| Step 1: Create Directory Structure                 | 30        |
| Step 2: Create All Files                           | 30        |
| Step 3: Create requirements.txt Files              | 31        |
| Step 4: Install Lambda Layer Dependencies          | 31        |
| Step 5: Set Up Python Virtual Environment          | 32        |
| Step 6: Set Environment Variables                  | 32        |
| Step 7: Configure the LLM Model ID                 | 32        |
| Deploy the Solution                                | 32        |
| Step 1: Bootstrap CDK (first time only)            | 32        |
| Step 2: Synthesize the CloudFormation Template     | 33        |
| Step 3: Deploy the Stack                           | 33        |
| Step 4: Verify                                     | 33        |
| Upload Documents and Build the Index               | 33        |
| Step 1: Get Bucket Name                            | 34        |
| Step 2: Upload Test Document                       | 34        |
| Step 3: Process the Document                       | 34        |
| Step 4: Build the FAISS Index                      | 35        |
| Step 5: Test Query                                 | 35        |
| Upload PDF Documents                               | 35        |
| Step 1: Install Script Dependencies                | 36        |
| Step 2: Upload Script                              | 36        |
| Step 3: Upload Documents                           | 36        |
| Step 4: Rebuild the Index                          | 36        |
| Step 5: Verify the Index                           | 37        |
| Troubleshooting                                    | 37        |
| Lambda Layer                                       | 37        |
| PDF Parsing                                        | 38        |
| When uploading files to S3, metadata must be ASCII | 38        |
| S3 event notifications are not retroactive         | 38        |
| Concurrency throttling under bulk ingest           | 38        |
| Indexing and S3 List Pagination                    | 39        |
| <b>Running Test Queries</b>                        | <b>40</b> |
| Multi year retrieval                               | 40        |
| Output                                             | 40        |
| Comments                                           | 41        |
| Resistance to false premises                       | 41        |
| Output                                             | 41        |
| Comments                                           | 42        |
| Comparing crypto frenzy with 2006-7 subprime       | 42        |
| Output                                             | 42        |
| Comments                                           | 43        |
| <b>Cost</b>                                        | <b>44</b> |

|                                             |           |
|---------------------------------------------|-----------|
| <b>Results and Discussion</b>               | <b>44</b> |
| Github                                      | 44        |
| Retrieval Quality and Groundedness          | 45        |
| Comparison with the Local Llama 2 Prototype | 45        |
| Features Proposed but not implemented       | 45        |
| Toward Agentic AI in AWS                    | 45        |
| Limitations                                 | 46        |

## Introduction and Purpose

This guide consolidates the design, implementation, debugging, and operational lessons learned while building a production-style Retrieval-Augmented Generation (RAG) system on AWS. The system ingests PDF documents (in this case, a corpus of approximately 3,300 financial press articles spanning 2020–2026), processes them into embeddings using Amazon Bedrock's Titan model, stores a FAISS vector index in Amazon S3, and answers natural-language questions through an API Gateway endpoint backed by Anthropic Claude on Bedrock.

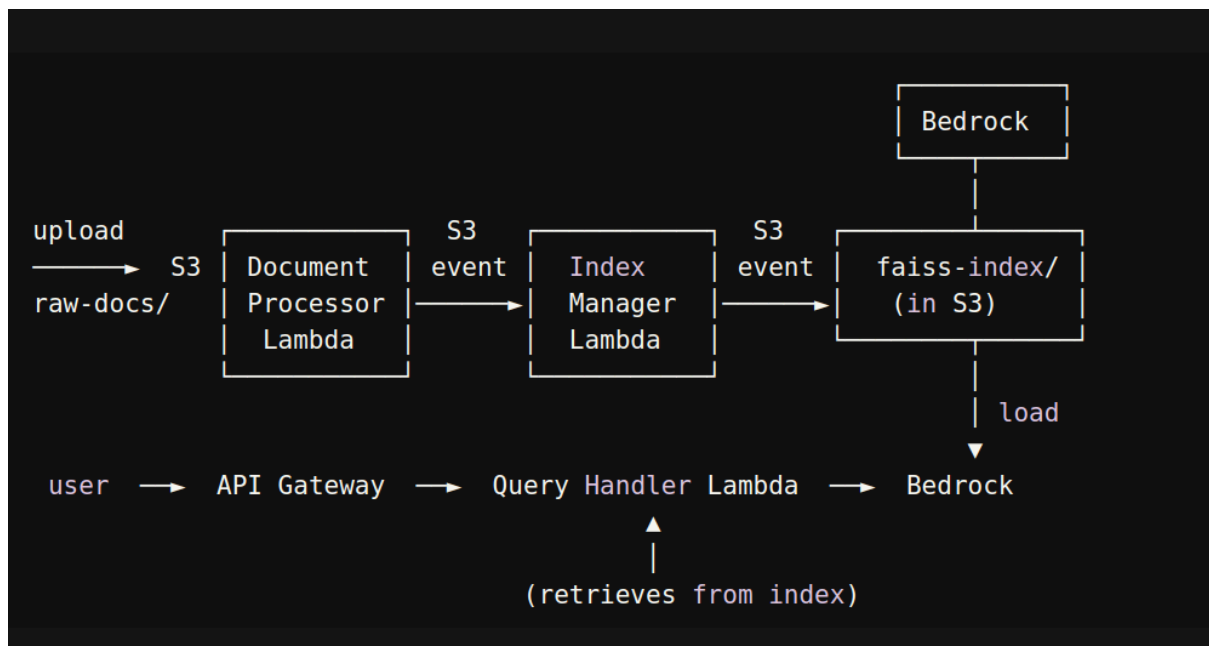
## Background: the local Llama 2 prototype

This AWS system is the cloud-native successor to an earlier local prototype ([Pareti, March 2025](#)) that implemented the same RAG pattern entirely on a Ubuntu workstation with an Nvidia GPU and 6 GB of VRAM. The prototype used PDF text extraction, embedding into a FAISS index, top-k semantic retrieval, and a two-step prompt strategy that first summarises the retrieved chunks and then generates a final answer combining document context with the model's general knowledge — but it did so under hardware constraints that conditioned subsequent decisions. The **Memory limit** was critical and conditioned design choices, even running Llama 2 7B in 4-bit quantization:

- a low dimension sentence transformer for embeddings;
- only the first 1000 characters of each document were indexed;
- lower output quality, inherent to a relatively small LLM compared to state-of-the-art models.

Further details are available in the original prototype document.

## Architecture Overview



## Part I: All Required Files

## Project Structure

```
rag-solution/
├── app.py
├── cdk.json
├── requirements.txt
├── config/
│   └── config.py
├── stacks/
│   ├── __init__.py
│   └── rag_stack.py
├── scripts/
│   ├── upload_documents_fixed.py
│   └── test_query.py
├── lambdas/
│   ├── document_processor/
│   │   └── document_processor.py
│   ├── index_manager/
│   │   └── index_manager.py
│   └── query_handler/
│       └── query_handler.py
└── layers/
    ├── dependencies/
    │   ├── requirements.txt
    │   └── python/
    │       ├── faiss/
    │       ├── numpy/
    │       └── pypdf/
```

## Root Files

### app.py

```
#!/usr/bin/env python3
import os
import aws_cdk as cdk
from stacks.rag_stack import RAGStack

app = cdk.App()
```

```

RAGStack(
    app,
    "RAGStack",
    env=cdk.Environment(
        account=os.getenv("CDK_DEFAULT_ACCOUNT"),
        region=os.getenv("CDK_DEFAULT_REGION", "us-east-1")
    ),
    description="RAG Solution with FAISS vector store and Bedrock"
)

app.synth()

```

## cdk.json

```

{
  "app": "python3 app.py",
  "watch": {
    "include": ["***"],
    "exclude": [
      "README.md",
      "cdk*.json",
      "requirements*.txt",
      "**/__pycache__",
      "**/.pytest_cache",
      "**/*.pyc",
      ".git",
      ".venv",
      "datasets"
    ]
  },
  "context": {
    "@aws-cdk/aws-lambda:recognizeLayerVersion": true,
    "@aws-cdk/core:stackRelativeExports": true,
    "@aws-cdk/aws-apigateway:usagePlanKeyOrderInsensitiveId": true,
    "@aws-cdk/core:enablePartitionLiterals": true,
    "@aws-cdk/aws-iam:minimizePolicies": true,
    "@aws-cdk/core:validateSnapshotRemovalPolicy": true
  }
}

```

## requirements.txt

```
aws-cdk-lib==2.170.0
```

```
constructs>=10.0.0,<11.0.0
python-dotenv==1.0.0
boto3==1.34.0
```

requirements-dev.txt

```
-r requirements.txt
pytest==7.4.0
pytest-cov==4.1.0
moto==4.2.0
```

.env.example

```
CDK_DEFAULT_ACCOUNT=123456789012
CDK_DEFAULT_REGION=us-east-1
```

## Config Directory

config/[config.py](#)

The system uses FAISS [IndexFlatIP](#) for exact inner-product search. Flat indexing is optimal for corpora under ~100k vectors, where exhaustive search is fast enough and recall is perfect. [IVFFlat](#) is noted as a future option for larger deployments.

```
from dataclasses import dataclass

@dataclass
class LambdaConfig:
    document_processor_memory: int = 1024
    document_processor_timeout: int = 300
    index_manager_memory: int = 3008 # FAISS needs more memory
```

```
index_manager_timeout: int = 600
query_handler_memory: int = 3008
query_handler_timeout: int = 30
```

```
@dataclass
class BedrockConfig:
    embedding_model_id: str = "amazon.titan-embed-text-v2:0"
    llm_model_id: str = "anthropic.claude-3-sonnet-20240229-v1:0"
    embedding_dimension: int = 1024
```

```
@dataclass
class ChunkingConfig:
    chunk_size: int = 1000
    chunk_overlap: int = 200
```

```
@dataclass
class FAISSConfig:
    index_file: str = "faiss_index.bin"
    metadata_file: str = "metadata.json"
    index_type: str = "IVFFlat" # Options: Flat, IVFFlat, HNSW
    nlist: int = 100 # Number of clusters for IVFFlat
```

```
@dataclass
class RAGConfig:
    lambda_config: LambdaConfig
    bedrock: BedrockConfig
    chunking: ChunkingConfig
    faiss: FAISSConfig

    @classmethod
    def default(cls) -> "RAGConfig":
        return cls(
            lambda_config=LambdaConfig(),
            bedrock=BedrockConfig(),
            chunking=ChunkingConfig(),
            faiss=FAISSConfig()
        )
```

```
def get_config() -> RAGConfig:
    return RAGConfig.default()
```

## Stacks Directory

stacks/ [\\_\\_init\\_\\_.py](#)

```
from .rag_stack import RAGStack

__all__ = ["RAGStack"]
```

stacks/rag\_stack.py

```
import aws_cdk.aws_s3_notifications as s3n
from aws_cdk import (
    Stack,
    Duration,
    RemovalPolicy,
    CfnOutput,
    aws_s3 as s3,
    aws_lambda as lambda_,
    aws_iam as iam,
    aws_apigateway as apigw,
)
from constructs import Construct

class RAGStack(Stack):
    def __init__(self, scope: Construct, construct_id: str, **kwargs) -> None:
        super().__init__(scope, construct_id, **kwargs)

        # S3 Bucket for documents and FAISS index
        self.bucket = s3.Bucket(
            self, "RAGBucket",
            removal_policy=RemovalPolicy.DESTROY,
            auto_delete_objects=True,
            versioned=True,
            encryption=s3.BucketEncryption.S3_MANAGED,
        )

        # Lambda Layer for shared dependencies
        dependencies_layer = lambda_.LayerVersion(
            self, "DependenciesLayer",
            code=lambda_.Code.from_asset("layers/dependencies"),
```

```

        compatible_runtimes=[lambda_.Runtime.PYTHON_3_9],
        description="FAISS, numpy, and other dependencies",
    )

    # Common environment variables
    common_env = {
        "BUCKET_NAME": self.bucket.bucket_name,
        "EMBEDDING_MODEL_ID": "amazon.titan-embed-text-v2:0",
        # "LLM_MODEL_ID": "us.anthropic.claude-3-5-haiku-20241022-v1:0",
        "LLM_MODEL_ID": "us.anthropic.claude-sonnet-4-5-20250929-v1:0",
        "INDEX_PREFIX": "faiss-index/",
    }

    # IAM Policy for Bedrock
    bedrock_policy = iam.PolicyStatement(
        actions=[
            "bedrock:InvokeModel",
            "bedrock:InvokeModelWithResponseStream",
        ],
        resources=["*"],
    )

    # Document Processor Lambda
    self.document_processor = lambda_.Function(
        self, "DocumentProcessor",
        runtime=lambda_.Runtime.PYTHON_3_9,
        handler="document_processor.handler",
        code=lambda_.Code.from_asset("lambdas/document_processor"),
        timeout=Duration.minutes(5),
        memory_size=1024,
        environment=common_env,
        layers=[dependencies_layer],
    )
    self.document_processor.add_to_role_policy(bedrock_policy)
    self.bucket.grant_read_write(self.document_processor)
    # Trigger Document Processor when files are uploaded
    #
    self.bucket.add_event_notification(s3.EventType.OBJECT_CREATED, s3n.LambdaDestination(
        self.document_processor), s3n.LambdaDestination(self.document_processor), s3.NotificationKeyFilter(
        prefix="uploads/"))
    self.bucket.add_event_notification(s3.EventType.OBJECT_CREATED,
        s3n.LambdaDestination(self.document_processor),
        s3.NotificationKeyFilter(prefix="uploads/"))

    # Index Manager Lambda
    self.index_manager = lambda_.Function(
        self, "IndexManager",

```

```

        runtime=lambda_.Runtime.PYTHON_3_9,
        handler="index_manager.handler",
        code=lambda_.Code.from_asset("lambdas/index_manager"),
        timeout=Duration.minutes(10),
        memory_size=3008,
        environment=common_env,
        layers=[dependencies_layer],
    )
    self.index_manager.add_to_role_policy(bedrock_policy)
    self.bucket.grant_read_write(self.index_manager)

    # Query Handler Lambda
    self.query_handler = lambda_.Function(
        self, "QueryHandler",
        runtime=lambda_.Runtime.PYTHON_3_9,
        handler="query_handler.handler",
        code=lambda_.Code.from_asset("lambdas/query_handler"),
        timeout=Duration.seconds(30),
        memory_size=3008,
        environment=common_env,
        layers=[dependencies_layer],
#         reserved_concurrent_executions=2,
    )
    self.query_handler.add_to_role_policy(bedrock_policy)
    self.bucket.grant_read(self.query_handler)

    # API Gateway
    api = apigw.RestApi(
        self, "RAGAPI",
        rest_api_name="RAG Service",
        description="RAG Query API",
        deploy_options=apigw.StageOptions(stage_name="prod"),
    )

    query_resource = api.root.add_resource("query")
    query_resource.add_method(
        "POST",
        apigw.LambdaIntegration(self.query_handler),
        api_key_required=True,
    )
    api_key = api.add_api_key("RAGApiKey", api_key_name="rag-default-key",
description="Default key for RAG API access",)
    usage_plan =
api.add_usage_plan("RAGUsagePlan",name="rag-default-plan",throttle=apigw.ThrottleSet
tings(rate_limit=2,burst_limit=5,),quota=apigw.QuotaSettings(limit=1000,period=apigw.Peri
od.DAY,))

```

```

usage_plan.add_api_key(api_key)
usage_plan.add_api_stage(stage=api.deployment_stage)
# Outputs
CfnOutput(self, "BucketName", value=self.bucket.bucket_name)
CfnOutput(self, "APIEndpoint", value=api.url)
CfnOutput(self, "DocumentProcessorArn",
value=self.document_processor.function_arn)
CfnOutput(self, "IndexManagerArn", value=self.index_manager.function_arn)
CfnOutput(self, "ApiKeyId", value=api_key.key_id, description="Use 'aws apigateway
get-api-key --api-key <id> --include-value' to retrieve",)

```

## Lambda Functions

lambdas/document\_processor/document\_processor.py

```

import json
import os
import boto3
import re
import io
from urllib.parse import unquote_plus

s3 = boto3.client("s3")
bedrock = boto3.client("bedrock-runtime")

BUCKET_NAME = os.environ["BUCKET_NAME"]
EMBEDDING_MODEL_ID = os.environ.get("EMBEDDING_MODEL_ID",
"amazon.titan-embed-text-v2:0")
CHUNK_SIZE = 1000
CHUNK_OVERLAP = 200

def chunk_text(text, chunk_size=CHUNK_SIZE, overlap=CHUNK_OVERLAP):
    chunks = []
    start = 0
    text = re.sub(r'\s+', ' ', text).strip()

    while start < len(text):
        end = start + chunk_size
        if end < len(text):
            last_period = text.rfind('.', start, end)

```

```

        last_newline = text.rfind('\n', start, end)
        break_point = max(last_period, last_newline)
        if break_point > start:
            end = break_point + 1

    chunk = text[start:end].strip()
    if chunk:
        chunks.append(chunk)
    start = end - overlap

return chunks

def get_embedding(text):
    response = bedrock.invoke_model(
        modelId=EMBEDDING_MODEL_ID,
        body=json.dumps({"inputText": text}),
        contentType="application/json",
    )
    result = json.loads(response["body"].read())
    return result["embedding"]

def handler(event, context):
    print(f"Received event: {json.dumps(event)}")

    try:
        # Handle S3 event notification format
        if "Records" in event:
            record = event["Records"][0]
            bucket = record["s3"]["bucket"]["name"]
            key = unquote_plus(record["s3"]["object"]["key"])
        else:
            # Direct invocation format
            bucket = event.get("bucket", BUCKET_NAME)
            key = event["key"]

        print(f"Processing file: s3://{bucket}/{key}")

        # Skip non-PDF files
        if not key.lower().endswith('.pdf'):
            print(f"Skipping non-PDF file: {key}")
            return {
                "statusCode": 200,
                "body": json.dumps({"message": "Skipped non-PDF file", "key": key}),
            }

        # Download PDF
        response = s3.get_object(Bucket=bucket, Key=key)

```

```

content = response["Body"].read()
print(f"Downloaded {len(content)} bytes")

# Extract text from PDF using pypdf
from pypdf import PdfReader
pdf_reader = PdfReader(io.BytesIO(content))
text = ""
for page in pdf_reader.pages:
    page_text = page.extract_text()
    if page_text:
        text += page_text + "\n"
print(f"Extracted {len(text)} characters from PDF")

if not text.strip():
    print(f"No text extracted from PDF: {key}")
    return {
        "statusCode": 200,
        "body": json.dumps({"message": "No text extracted from PDF", "key": key}),
    }

chunks = chunk_text(text)
print(f"Created {len(chunks)} chunks")

processed_chunks = []
for i, chunk in enumerate(chunks):
    embedding = get_embedding(chunk)
    processed_chunks.append({
        "chunk_id": f"{key}_{i}",
        "text": chunk,
        "embedding": embedding,
        "source": key,
        "chunk_index": i,
    })

output_key = f"processed/{key.replace('/', '_')}.json"
s3.put_object(
    Bucket=bucket,
    Key=output_key,
    Body=json.dumps(processed_chunks),
    ContentType="application/json",
)
print(f"Saved processed chunks to {output_key}")

return {
    "statusCode": 200,
    "body": json.dumps({
        "message": "Document processed successfully",
    })
}

```

```

        "chunks_created": len(processed_chunks),
        "output_key": output_key,
    }},
}
except Exception as e:
    print(f"Error processing document: {str(e)}")
    import traceback
    traceback.print_exc()
    raise

```

lambdas/document\_processor/requirements.txt

# Dependencies provided by layer

lambdas/index\_manager/index\_manager.py

```

import json
import os
import boto3
import numpy as np
import faiss
import tempfile

s3 = boto3.client("s3")

BUCKET_NAME = os.environ["BUCKET_NAME"]
INDEX_PREFIX = os.environ.get("INDEX_PREFIX", "faiss-index/")

def handler(event, context):
    try:
        action = event.get("action", "build")

        if action == "build":
            return build_index(event)
        elif action == "update":
            return update_index(event)
        else:
            return {"statusCode": 400, "body": json.dumps({"error": "Invalid action"})}
    except Exception as e:
        return {"statusCode": 500, "body": json.dumps({"error": str(e)})}

```

```

def build_index(event):
    processed_prefix = event.get("processed_prefix", "processed/")

    all_embeddings = []
    all_metadata = []

    # Paginate through all objects
    paginator = s3.get_paginator('list_objects_v2')
    pages = paginator.paginate(Bucket=BUCKET_NAME, Prefix=processed_prefix)

    for page in pages:
        for obj in page.get("Contents", []):
            key = obj["Key"]
            if key.endswith(".json"):
                data = s3.get_object(Bucket=BUCKET_NAME, Key=key)
                chunks = json.loads(data["Body"].read().decode("utf-8"))

                for chunk in chunks:
                    all_embeddings.append(chunk["embedding"])
                    all_metadata.append({
                        "chunk_id": chunk["chunk_id"],
                        "text": chunk["text"],
                        "source": chunk["source"],
                        "chunk_index": chunk["chunk_index"],
                    })

    if not all_embeddings:
        return {"statusCode": 400, "body": json.dumps({"error": "No embeddings found"})}

    embeddings_array = np.array(all_embeddings, dtype=np.float32)
    dimension = embeddings_array.shape[1]

    index = faiss.IndexFlatIP(dimension)
    faiss.normalize_L2(embeddings_array)
    index.add(embeddings_array)

    with tempfile.TemporaryDirectory() as tmpdir:
        index_path = os.path.join(tmpdir, "index.faiss")
        metadata_path = os.path.join(tmpdir, "metadata.json")

        faiss.write_index(index, index_path)
        with open(metadata_path, "w") as f:
            json.dump(all_metadata, f)

    s3.upload_file(index_path, BUCKET_NAME, f'{INDEX_PREFIX}index.faiss')
    s3.upload_file(metadata_path, BUCKET_NAME, f'{INDEX_PREFIX}metadata.json')

```

```

return {
    "statusCode": 200,
    "body": json.dumps({
        "message": "Index built successfully",
        "total_vectors": len(all_embeddings),
        "dimension": dimension,
    }),
}

def update_index(event):
    return build_index(event)

```

lambdas/index\_manager/requirements.txt

# Dependencies provided by layer

lambdas/query\_handler/query\_handler.py

```

import json
import os
import boto3
import numpy as np
import faiss
import tempfile

s3 = boto3.client("s3")
bedrock = boto3.client("bedrock-runtime")

BUCKET_NAME = os.environ["BUCKET_NAME"]
INDEX_PREFIX = os.environ.get("INDEX_PREFIX", "faiss-index/")
EMBEDDING_MODEL_ID = os.environ.get("EMBEDDING_MODEL_ID",
    "amazon.titan-embed-text-v2:0")
LLM_MODEL_ID = os.environ.get("LLM_MODEL_ID",
    "anthropic.claude-3-sonnet-20240229-v1:0")

index_cache = {}

def get_embedding(text):
    response = bedrock.invoke_model(
        modelId=EMBEDDING_MODEL_ID,

```

```

        body=json.dumps({"inputText": text}),
        contentType="application/json",
    )
    result = json.loads(response["body"].read())
    return result["embedding"]

def load_index():
    if "index" in index_cache and "metadata" in index_cache:
        return index_cache["index"], index_cache["metadata"]

    with tempfile.TemporaryDirectory() as tmpdir:
        index_path = os.path.join(tmpdir, "index.faiss")
        metadata_path = os.path.join(tmpdir, "metadata.json")

        s3.download_file(BUCKET_NAME, f'{INDEX_PREFIX}index.faiss', index_path)
        s3.download_file(BUCKET_NAME, f'{INDEX_PREFIX}metadata.json',
            metadata_path)

        index = faiss.read_index(index_path)
        with open(metadata_path, "r") as f:
            metadata = json.load(f)

        index_cache["index"] = index
        index_cache["metadata"] = metadata

    return index, metadata

def search(query, k=5):
    index, metadata = load_index()

    query_embedding = np.array([get_embedding(query)], dtype=np.float32)
    faiss.normalize_L2(query_embedding)

    distances, indices = index.search(query_embedding, k)

    results = []
    for i, idx in enumerate(indices[0]):
        if idx < len(metadata):
            result = metadata[idx].copy()
            result["score"] = float(distances[0][i])
            results.append(result)

    return results

def generate_response(query, context):
    context_text = "\n\n".join([f"[Source: {c['source']}] \n{c['text']}" for c in context])

```

```
prompt = f""Based on the following context, answer the question. If the answer is not in
the context, say so.
```

```
Context:
{context_text}
```

```
Question: {query}
```

```
Answer: ""
```

```
response = bedrock.invoke_model(
    modelId=LLM_MODEL_ID,
    body=json.dumps({
        "anthropic_version": "bedrock-2023-05-31",
        "max_tokens": 1024,
        "messages": [{"role": "user", "content": prompt}],
    }),
    contentType="application/json",
)
```

```
result = json.loads(response["body"].read())
return result["content"][0]["text"]
```

```
def handler(event, context):
    try:
        body = json.loads(event.get("body", "{}"))
        query = body.get("query")
        k = body.get("k", 5)

        if not query:
            return {
                "statusCode": 400,
                "body": json.dumps({"error": "Query is required"}),
            }

        search_results = search(query, k)
        response_text = generate_response(query, search_results)

        return {
            "statusCode": 200,
            "headers": {"Content-Type": "application/json"},
            "body": json.dumps({
                "query": query,
                "answer": response_text,
                "sources": [{"source": r["source"], "score": r["score"]} for r in search_results],
            }),
        }
```

```
except Exception as e:
    return {
        "statusCode": 500,
        "body": json.dumps({"error": str(e)}),
    }
```

lambdas/query handler/requirements.txt

# Dependencies provided by layer

## Layers Directory

layers/dependencies/requirements.txt

```
faiss-cpu==1.7.4
numpy==1.26.4
pypdf==3.17.0
```

```
layers/dependencies/
├── requirements.txt
├── python/
│   ├── faiss/      # Vector similarity search
│   ├── numpy/     # Numerical operations
│   └── pypdf/      # PDF text extraction
```

Installation:

```
cd layers/dependencies
pip install -r requirements.txt -t python/ --platform manylinux2014_x86_64
--only-binary=:all:
```

## Scripts Directory

## scripts/upload\_documents\_fixed.py

```
#!/usr/bin/env python3
"""
Script to bulk upload PDF documents to the RAG system.
Uploads to a FLAT S3 structure (all files in uploads/ folder).

Usage:
python upload_documents.py /path/to/datasets
python upload_documents.py /path/to/datasets --dry-run
python upload_documents.py /path/to/datasets --year 2024
python upload_documents.py /path/to/datasets --from-year 2023 --to-year 2025
"""

import os
import sys
import boto3
import argparse
from pathlib import Path
from collections import defaultdict
import time
import uuid

VALID_YEARS = [str(year) for year in range(2020, 2027)]

MONTH_NAMES = {
    'january': '01', 'jan': '01', '01': '01',
    'february': '02', 'feb': '02', '02': '02',
    'march': '03', 'mar': '03', '03': '03',
    'april': '04', 'apr': '04', '04': '04',
    'may': '05', '05': '05',
    'june': '06', 'jun': '06', '06': '06',
    'july': '07', 'jul': '07', '07': '07',
    'august': '08', 'aug': '08', '08': '08',
    'september': '09', 'sep': '09', 'sept': '09', '09': '09',
    'october': '10', 'oct': '10', '10': '10',
    'november': '11', 'nov': '11', '11': '11',
    'december': '12', 'dec': '12', '12': '12',
}

MONTH_DISPLAY = {
    '01': 'january', '02': 'february', '03': 'march', '04': 'april',
    '05': 'may', '06': 'june', '07': 'july', '08': 'august',
    '09': 'september', '10': 'october', '11': 'november', '12': 'december',
}
```

```

def sanitize_metadata(text):
    """Convert text to ASCII-safe string for S3 metadata."""
    return text.encode('ascii', 'replace').decode('ascii')

def get_bucket_name():
    """Get the documents bucket name from CloudFormation outputs or environment."""
    # Check environment variable first
    bucket = os.environ.get('DOCUMENTS_BUCKET')
    if bucket:
        return bucket

    cf_client = boto3.client("cloudformation")
    try:
        response = cf_client.describe_stacks(StackName="RAGStack")
        outputs = response["Stacks"][0]["Outputs"]
        for output in outputs:
            if output["OutputKey"] == "BucketName":
                return output["OutputValue"]
        raise ValueError("BucketName not found in stack outputs")
    except Exception as e:
        print(f"Error getting bucket name: {e}")
        print("Make sure the RAGStack is deployed or set DOCUMENTS_BUCKET env
var.")
        sys.exit(1)

def parse_year_month(filepath):
    """Extract year and month from file path."""
    parts = filepath.parts
    year = None
    month_num = None
    month_name = None

    for part in parts:
        part_clean = part.strip()
        if part_clean in VALID_YEARS:
            year = part_clean
        part_lower = part_clean.lower()
        if part_lower in MONTH_NAMES:
            month_num = MONTH_NAMES[part_lower]
            month_name = MONTH_DISPLAY[month_num]

    return year, month_num, month_name

```

```

def discover_files(base_path, from_year=2020, to_year=2026):
    """Discover all PDF files organized by year/month."""
    files_by_period = defaultdict(list)
    unorganized_files = []
    skipped_files = []

    all_pdfs = list(base_path.glob("**/*.pdf"))

    for pdf_path in all_pdfs:
        year, month_num, month_name = parse_year_month(pdf_path)

        if year and month_num:
            year_int = int(year)
            if year_int < from_year or year_int > to_year:
                skipped_files.append((pdf_path, year, month_name, "outside year range"))
                continue
            files_by_period[(year, month_num, month_name)].append(pdf_path)
        else:
            unorganized_files.append(pdf_path)

    return dict(files_by_period), unorganized_files, skipped_files

def upload_file(s3_client, bucket, filepath, year=None, month=None, month_name=None):
    """Upload a single PDF file to S3 with FLAT structure."""
    file_uuid = str(uuid.uuid4())[0:8]
    key = f"uploads/{file_uuid}_{filepath.name}"

    metadata = {
        "original-filename": sanitize_metadata(filepath.name),
        "original-path": sanitize_metadata(str(filepath)),
        "upload-id": file_uuid,
    }

    if year:
        metadata["source-year"] = year
    if month:
        metadata["source-month"] = month
    if month_name:
        metadata["source-month-name"] = month_name

    try:
        file_size = filepath.stat().st_size
        s3_client.upload_file(
            str(filepath),
            bucket,

```

```

        key,
        ExtraArgs={"ContentType": "application/pdf", "Metadata": metadata}
    )
    return {"file": filepath.name, "status": "success", "size": file_size, "year": year or "N/A"}
except Exception as e:
    return {"file": filepath.name, "status": "error", "error": str(e), "year": year or "N/A"}

def format_size(size_bytes):
    """Format file size in human readable format."""
    for unit in ['B', 'KB', 'MB', 'GB']:
        if size_bytes < 1024:
            return f"{size_bytes:.1f} {unit}"
        size_bytes /= 1024
    return f"{size_bytes:.1f} TB"

def print_discovery_summary(files_by_period, unorganized, skipped, from_year, to_year):
    """Print summary of discovered files."""
    print("\n" + "=" * 70)
    print(f"DISCOVERY SUMMARY (Years {from_year}-{to_year})")
    print("=" * 70)

    total_size = 0
    total_files = 0

    if files_by_period:
        sorted_periods = sorted(files_by_period.keys(), key=lambda x: (x[0], x[1]))
        current_year = None
        year_file_count = 0
        year_size = 0

        for year, month_num, month_name in sorted_periods:
            if year != current_year:
                if current_year is not None:
                    print(f"    ____ Year total: {year_file_count} file(s) ({format_size(year_size)})\n")
                    print(f"    📁 {year}/")
                current_year = year
                year_file_count = 0
                year_size = 0

            files = files_by_period[(year, month_num, month_name)]
            count = len(files)
            size = sum(f.stat().st_size for f in files)
            total_size += size
            total_files += count
            year_file_count += count

```

```

        year_size += size
        print(f"    |—— {month_name:<12} {count:>4} file(s) ({format_size(size):>10})")

    if current_year is not None:
        print(f"    |—— Year total: {year_file_count} file(s) ({format_size(year_size)})")

    print(f"\n📁 Total organized files: {total_files} ({format_size(total_size)})")

    if unorganized:
        unorg_size = sum(f.stat().st_size for f in unorganized)
        print(f"\n⚠️ Unorganized files: {len(unorganized)} ({format_size(unorg_size)})")
        for f in unorganized[:5]:
            print(f"    - {f.name}")
        if len(unorganized) > 5:
            print(f"    ... and {len(unorganized) - 5} more")

    print(f"\n📦 Total size to upload: {format_size(total_size)}")
    print("=" * 70)

    return total_files, total_size

def upload_documents(path, dry_run=False, from_year=2020, to_year=2026,
include_unorganized=False):
    """Upload PDF documents from year/month directory structure to FLAT S3 structure."""
    base_path = Path(path)

    if not base_path.exists():
        print(f"Error: Path does not exist: {base_path}")
        sys.exit(1)

    if not base_path.is_dir():
        print(f"Error: Path is not a directory: {base_path}")
        sys.exit(1)

    print(f"🔍 Scanning directory: {base_path}")
    print(f"    Looking for: years {from_year}-{to_year}, all 12 months")

    files_by_period, unorganized_files, skipped_files = discover_files(base_path, from_year,
to_year)

    if not files_by_period and not unorganized_files:
        print("\nNo PDF files found matching criteria.")
        return

    total_files, total_size = print_discovery_summary(
        files_by_period, unorganized_files, skipped_files, from_year, to_year

```

```

)

if include_unorganized:
    total_files += len(unorganized_files)

if total_files == 0:
    print("\nNo files to upload with current filters.")
    return

if dry_run:
    print(f"\n ♦ DRY RUN - Would upload {total_files} file(s)")
    return

print(f"\n📁 Ready to upload {total_files} file(s)")
response = input("Proceed? [y/N]: ").strip().lower()
if response != 'y':
    print("Upload cancelled.")
    return

bucket = get_bucket_name()
print(f"\n☁ Uploading to: s3://{bucket}/uploads/")

s3_client = boto3.client("s3")

# Build upload list
upload_tasks = []
for (year, month_num, month_name), files in files_by_period.items():
    for filepath in files:
        upload_tasks.append({"filepath": filepath, "year": year, "month": month_num,
"month_name": month_name})

if include_unorganized:
    for filepath in unorganized_files:
        upload_tasks.append({"filepath": filepath, "year": None, "month": None,
"month_name": None})

# Upload sequentially
results = []
start_time = time.time()
successful = 0
failed = 0
total_uploaded_size = 0

print(f"\nUploading {len(upload_tasks)} files...\n")

for i, task in enumerate(upload_tasks, 1):
    result = upload_file(s3_client, bucket, task["filepath"], task["year"], task["month"],

```

```

task["month_name"])
    results.append(result)

    if result["status"] == "success":
        successful += 1
        total_uploaded_size += result.get("size", 0)
        icon = "✓"
    else:
        failed += 1
        icon = "✗"
        print(f" {icon} {result['file']}: {result.get('error', 'Unknown error')}")

progress = (i / len(upload_tasks)) * 100
print(f"\r[{progress:5.1f}%] {successful} uploaded, {failed} failed", end="", flush=True)

elapsed = time.time() - start_time

# Summary
print("\n\n" + "=" * 70)
print("UPLOAD COMPLETE")
print("=" * 70)
print(f"🕒 Duration: {elapsed:.1f} seconds")
print(f"✅ Successful: {successful} ({format_size(total_uploaded_size)})")
print(f"❌ Failed: {failed}")

if failed > 0:
    print("\nFailed files:")
    for result in results:
        if result["status"] == "error":
            print(f" - {result['file']}: {result['error']}")

print("=" * 70)

def main():
    parser = argparse.ArgumentParser(description="Upload PDF documents to RAG system")
    parser.add_argument("path", help="Path to directory containing year/month structured PDFs")
    parser.add_argument("--dry-run", action="store_true", help="Preview without uploading")
    parser.add_argument("--year", type=int, choices=range(2020, 2027), metavar="YEAR", help="Upload only specific year")
    parser.add_argument("--from-year", type=int, default=2020, metavar="YEAR", help="Start year (default: 2020)")
    parser.add_argument("--to-year", type=int, default=2026, metavar="YEAR", help="End year (default: 2026)")

```

```

    parser.add_argument("--include-unorganized", action="store_true", help="Include files
not in year/month structure")

    args = parser.parse_args()

    from_year = args.from_year
    to_year = args.to_year

    if args.year:
        from_year = args.year
        to_year = args.year

    if from_year > to_year:
        print("Error: --from-year cannot be greater than --to-year")
        sys.exit(1)

    upload_documents(
        path=args.path,
        dry_run=args.dry_run,
        from_year=from_year,
        to_year=to_year,
        include_unorganized=args.include_unorganized
    )

if __name__ == "__main__":

```

## Test the script

Before loading the files to S3, one may want to test the script with the following option:

```
$ python scripts/upload_documents_fixed.py ./datasets --dry-run
```

## Part II: Deployment Guide

The development workstation runs Ubuntu and requires Python 3.9, Node.js 18+, the AWS CLI v2, Docker (used by CDK for asset bundling), and the AWS CDK CLI installed globally via `npm install -g aws-cdk`. AWS credentials are configured via `aws configure` after generating an access key pair from the IAM console; the recommended verification command is `aws sts get-caller-identity`, which should return the account ID and IAM ARN. If that call returns `InvalidClientTokenId`, the access key has either been rotated or never propagated, and a new key must be generated.

## Prerequisites on Ubuntu PC

### Step 1: Install Required Software

```
# Update package list
sudo apt update

# Install Python 3.9 - must match Lambda runtime
sudo apt install -y software-properties-common
sudo add-apt-repository -y ppa:deadsnakes/ppa
sudo apt update
sudo apt install -y python3.9 python3.9-venv python3.9-dev

# Install Node.js 18+
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs

# Install AWS CLI v2
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
rm -rf awscliv2.zip aws/

# Install unzip if not present
sudo apt install -y unzip
```

### Step 2: Install AWS CDK

```
sudo npm install -g aws-cdk
cdk --version
```

### Step 3: Configure AWS Credentials

```
aws configure
```

Enter your AWS Access Key ID, Secret Access Key, default region (us-east-1), and output format (json).

verify credentials:

```
aws sts get-caller-identity
```

## Prerequisites in AWS Console

In the Bedrock console, model access must be explicitly granted for "Amazon Titan Embeddings V2" and for the chosen Claude model. Newer Claude models such as Claude Sonnet 4.5 are only accessible through an *inference profile* (the `us.anthropic. ...` prefix), not as a bare foundation-model ARN.

### Step 4: Enable Bedrock Model Access

Log into the AWS Console and navigate to Amazon Bedrock, then Model access. Request access to "Amazon Titan Embeddings V2" and "Anthropic Claude 4.5 Sonnet".

To find available models:

```
aws bedrock list-foundation-models --by-provider anthropic \
  --query "modelSummaries[*].[modelId,modelLifecycle.status]" --output table
```

Select a model with status ACTIVE. As of this writing, the working model ID is `us.anthropic.claude-sonnet-4-5-20250929-v1:0`.

## Create the Project

### Step 1: Create Directory Structure

```
mkdir -p rag-solution/{config,stacks,scripts}
mkdir -p rag-solution/lambdas/{document_processor,index_manager,query_handler}
mkdir -p rag-solution/layers/dependencies/python/
cd rag-solution
```

### Step 2: Create All Files

Create each file from Part I above.

### Step 3: Create requirements.txt Files

Project root requirements.txt:

```
aws-cdk-lib==2.170.0
constructs>=10.0.0,<11.0.0
python-dotenv==1.0.0
boto3==1.34.0
```

layers/dependencies/requirements.txt:

```
faiss-cpu==1.7.4
numpy==1.26.4
pypdf==3.17.0
```

### Step 4: Install Lambda Layer Dependencies

*This step is critical. Lambda runs on Amazon Linux with Python 3.9, so dependencies (NumPy, FAISS) must be compiled for that exact platform and Python minor version.*

```
cd layers/dependencies
rm -rf python/*

pip install \
  --platform manylinux2014_x86_64 \
  --implementation cp \
  --python-version 3.9 \
  --only-binary=:all: \
  --target python/ \
  numpy==1.26.4 faiss-cpu==1.7.4 pypdf==3.17.0

# Verify the installation
ls python/
```

The four flags (`--platform`, `--implementation`, `--python-version`, `--only-binary`) collectively force `pip` to download the precompiled Linux wheel that matches the Lambda runtime rather than building from source against the local interpreter. The Lambda function definitions in `rag_stack.py` accordingly specify `lambda_.Runtime.PYTHON_3_9`.

## Step 5: Set Up Python Virtual Environment

```
cd ~/rag-solution
python3.9 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

## Step 6: Set Environment Variables

```
# Get your AWS account ID
AWS_ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
echo "Your AWS Account ID: $AWS_ACCOUNT"

# Create .env file
cat > .env << EOF
CDK_DEFAULT_ACCOUNT=$AWS_ACCOUNT
CDK_DEFAULT_REGION=us-east-1
EOF

# Load environment variables
export $(cat .env | xargs)
```

## Step 7: Configure the LLM Model ID

Edit `stacks/rag_stack.py` and set the correct model ID in the Lambda environment variables:

```
"LLM_MODEL_ID": "us.anthropic.claude-sonnet-4-5-20250929-v1:0",
```

## Deploy the Solution

### Step 1: Bootstrap CDK (first time only)

The Python virtual environment is created and populated with the CDK dependencies (`aws-cdk-lib==2.170.0`, `constructs>=10.0.0,<11.0.0`, `boto3==1.34.0`, `python-dotenv==1.0.0`).

```
cd ~/rag-solution
source .venv/bin/activate
cdk bootstrap aws://$CDK_DEFAULT_ACCOUNT/$CDK_DEFAULT_REGION
```

## Step 2: Synthesize the CloudFormation Template

```
cdk synth
```

This command produces the CloudFormation template.

## Step 3: Deploy the Stack

```
cdk deploy
```

This command provisions the stack.

Output it like :

```
RAGStack.APIEndpoint = https://xxxxxxxxx.execute-api.us-east-1.amazonaws.com/prod/
RAGStack.BucketName = ragstack-ragbucketxxxxxxxx-xxxxxxxxxxxxx
```

## Step 4: Verify

```
# Get stack outputs
aws cloudformation describe-stacks --stack-name RAGStack \
  --query "Stacks[0].Outputs" --output table

# List Lambda functions
aws lambda list-functions --query "Functions[?contains(FunctionName,
'RAGStack')].FunctionName" --output table
```

## Upload Documents and Build the Index

## Step 1: Get Bucket Name

```
BUCKET=$(aws cloudformation describe-stacks --stack-name RAGStack \
  --query "Stacks[0].Outputs[?contains(OutputKey, 'Bucket')].OutputValue" --output text)
echo "Bucket: $BUCKET"
```

## Step 2: Upload Test Document

```
# Create a test document
echo "Amazon Bedrock is a fully managed service that offers foundation models from
leading AI companies. It provides access to models from Anthropic, AI21 Labs, Stability
AI, and Amazon. You can use it for text generation, summarization, and embeddings." >
test-doc.txt

# Upload to S3
aws s3 cp test-doc.txt s3://$BUCKET/uploads/

# Verify upload
aws s3 ls s3://$BUCKET/uploads/
```

## Step 3: Process the Document

Manual processing:

```
PROCESSOR=$(aws lambda list-functions \
  --query "Functions[?contains(FunctionName, 'DocumentProcessor')].FunctionName"
--output text)

aws lambda invoke \
  --function-name $PROCESSOR \
  --payload '{"bucket": "$BUCKET", "key": "uploads/test-doc.txt"}' \
  --cli-binary-format raw-in-base64-out \
  response.json

cat response.json
```

## Step 4: Build the FAISS Index

This step is required after uploading documents. The index must be rebuilt to make documents searchable.

```
INDEX_MANAGER=$(aws lambda list-functions \
  --query "Functions[?contains(FunctionName, 'IndexManager')].FunctionName" --output
  text)

aws lambda invoke \
  --function-name $INDEX_MANAGER \
  --payload '{}' \
  --cli-binary-format raw-in-base64-out \
  --cli-read-timeout 300 \
  response.json

cat response.json
```

expected output:

```
{"statusCode": 200, "body": "{\"message\": \"Index built successfully\", \"total_vectors\": 1,
  \"dimension\": 1024}\""}
```

## Step 5: Test Query

```
API_ENDPOINT=$(aws cloudformation describe-stacks --stack-name RAGStack \
  --query "Stacks[0].Outputs[?contains(OutputKey, 'APIEndpoint')].OutputValue" --output
  text)

curl -X POST ${API_ENDPOINT}query \
  -H "Content-Type: application/json" \
  -d '{"query": "What is Amazon Bedrock?"}'
```

## Upload PDF Documents

## Step 1: Install Script Dependencies

No additional dependencies required.

## Step 2: Upload Script

The source code is in Part 1 of this guide.

The code `scripts/upload_documents_fixed.py` traverses a local directory tree organised as `datasets/<year>/<month>/*.pdf`, validates that years fall in the supported 2020–2026 range, and uploads each file with a unique key of the form `uploads/<8-char-uuid>_<filename>.pdf`. Original path information is preserved in the S3 object metadata (`original-filename`, `original-path`, `source-year`, `source-month`, `source-month-name`).

**S3 metadata must be ASCII.** Many real-world article titles contain typographic Unicode characters — curly apostrophes ( `'` ), em-dashes ( `–` ), accented letters — and any of these in a metadata value causes `boto3` to raise a `Parameter validation failed: Non ascii characters found in S3 metadata` error. The misleading aspect of this failure is that the error surfaces during the PUT call and is often misreported as a "malformed PDF" problem.

## Step 3: Upload Documents

```
python scripts/upload_documents_fixed.py /path/to/pdf/files $BUCKET 2020
```

## Step 4: Rebuild the Index

```
aws lambda invoke \  
  --function-name $INDEX_MANAGER \  
  --payload '{}' \  
  --cli-binary-format raw-in-base64-out \  
  --cli-read-timeout 600 \  
  response.json
```

Once the bucket trigger has been added to the stack

```
(self.bucket.add_event_notification(s3.EventType.OBJECT_CREATED,  
s3n.LambdaDestination(self.document_processor),  
s3.NotificationKeyFilter(prefix="uploads/"))),
```

only *new* uploads cause the Document Processor to run. Files that were already in the bucket before the trigger was wired up will sit there untouched, and CloudWatch will show no invocations. The only

remedies are to either delete and re-upload the affected objects, or to invoke the processor manually for each existing key.

## Step 5: Verify the Index

```
# Check number of indexed documents
aws s3 cp s3://$BUCKET/faiss-index/metadata.json - | \
python3 -c "import sys,json; d=json.load(sys.stdin); \
print('Total vectors:', len(d)); \
sources=set(item.get('source','') for item in d); \
print('Unique sources:', len(sources))"
```

## Troubleshooting

Refer to [this document](#) that explains my own implementation. Below some key points are summarized.

### Lambda Layer

Lambda runs on Amazon Linux 2 with a specific Python interpreter, and any compiled C extension (which includes both NumPy and FAISS) must be built for that exact platform and Python minor version. Installing packages with a plain `pip install -t python/` on an Ubuntu workstation produces wheels that fail at import time inside Lambda with `No module named 'numpy.core._multiarray_umath'`.

To make it work do:

```
cd layers/dependencies
rm -rf python/*
pip install \
  "numpy==1.26.4" \
  "faiss-cpu==1.7.4" \
  "pypdf==3.17.0" \
  -t python/ \
  --platform manylinux2014_x86_64 \
  --implementation cp \
  --python-version 3.9 \
```

```
--only-binary=:all:
```

## PDF Parsing

The original design used PyMuPDF (`fitz`), which has complex native dependencies and produces a misleading `ImportError: cannot import name '_extra' from partially initialized module 'fitz' inside Lambda`. Switching to the pure-Python `pypdf` library eliminated the problem at the cost of less robust text extraction.

## When uploading files to S3, metadata must be ASCII

Many article titles contain typographic Unicode characters — curly apostrophes (`'`), em-dashes (`—`), accented letters — and any of these in a metadata value causes `boto3` to raise a `Parameter validation failed: Non ascii characters found in S3 metadata` error. The misleading aspect of this failure is that the error surfaces during the PUT call and is often misreported as a "malformed PDF" problem. The script has been reworked to fix the issue.

## S3 event notifications are not retroactive

Once the bucket trigger has been added to the stack (`self.bucket.add_event_notification(s3.EventType.OBJECT_CREATED, s3n.LambdaDestination(self.document_processor), s3.NotificationKeyFilter(prefix="uploads/"))`), only *new* uploads cause the Document Processor to run. Files that were already in the bucket before the trigger was wired up will sit there untouched, and CloudWatch will show no invocations. The only remedies are to either delete and re-upload the affected objects, or to invoke the processor manually for each existing key.

## Concurrency throttling under bulk ingest

An AWS account ships with a limit of **10 concurrent Lambda executions**. Uploading 3,477 PDFs may saturate that limit: every S3 PUT fires a Document Processor invocation, and once concurrency is exhausted both new invocations *and* the user-facing Query Handler are throttled with `TooManyRequestsException: Rate Exceeded`. From the outside this

presents as **Internal server error** on the `/query` endpoint, while the document corpus appears to load successfully.

## Indexing and S3 List Pagination

After the full 2020–2026 corpus was uploaded and the index was rebuilt, the index reported **15,812 vectors**. This number is suspicious: the 2020-only index had contained 8,093 vectors, and with seven years of roughly equal-sized data one would expect something closer to 50,000 vectors. A direct count of unique sources in the metadata file revealed the truth

```
aws s3 cp s3://<bucket>/faiss-index/metadata.json - | python3 -c \
"import sys,json; d=json.load(sys.stdin); \
print('Total entries:', len(d)); \
print('Unique sources:', len(set(item.get('source', '') for item in d)))"
```

```
Total entries: 15812
Unique sources: 1000
```

Exactly 1,000 unique source documents — which is the precise default **MaxKeys** value of the S3 `list_objects_v2` API. The original `index_manager.py` made a single un-paginated call:

```
response = s3.list_objects_v2(Bucket=BUCKET_NAME, Prefix=processed_prefix)
for obj in response.get("Contents", []):
    ...
```

This caps the index at the first 1,000 processed files **without warnings**; the remaining 2,281 documents in the corpus were never embedded into the searchable index. The fix is to use the paginator:

```
paginator = s3.get_paginator('list_objects_v2')
pages = paginator.paginate(Bucket=BUCKET_NAME, Prefix=processed_prefix)
for page in pages:
    for obj in page.get("Contents", []):
```

After deploying the corrected `index_manager.py` and rebuilding, the index grew to **50,727 vectors across 3,281 unique sources**, matching the number of processed files. Subsequent queries about post-pandemic monetary policy returned answers grounded in sources from across all seven years (Barron's and WSJ articles dated 2022 through 2025), confirming that the full corpus was now searchable.

## Running Test Queries

The following queries exercise different behaviours of the system.

### Multi year retrieval

```
curl -X POST https://b37trqi2xg.execute-api.us-east-1.amazonaws.com/prod/query -H
"Content-Type: application/json" -d '{"query": "Federal Reserve interest rate hikes in 2022
and 2023 to fight inflation", "k": 10}'
```

### Output

```
{
  "query": "Federal Reserve interest rate hikes in 2022 and 2023 to fight inflation",
  "answer": "Based on the context provided, here is information about Federal Reserve interest rate hikes in 2022 and 2023 to fight inflation:\n\n**2022:**\n- Fed officials projected three interest rate rises in 2022\n- The Fed raised rates for the first time since 2018 in March 2022 with a quarter-point (0.25%) increase\n- The Fed signaled as many as six additional rate increases for 2022 beyond the initial March hike\n- Officials expected rates to rise to at least 1.875% by the end of 2022\n- By October 2022, the Fed was implementing 0.75 percentage point increases\n\n**2023:**\n- Officials projected rates would reach 2.75% by the end of 2023\n- The context mentions the Fed raised rates \"at the most rapid pace in decades in 2022 and 2023 to combat high inflation\"\n\n**Purpose:**\n- The rate increases were implemented to:\n  - Combat high inflation\n  - End emergency pandemic stimulus\n  - Slow economic activity by curbing spending, hiring, and investment through higher borrowing costs and lower asset prices\n  - Reduce demand and lower inflation over time\n\nThe documents indicate the Fed wanted to taper its asset purchase program before lifting rates from zero to prevent inflation from staying too high.",
  "sources": [
    {
      "source": "uploads/10911517_Covid-19 Variant Upends Investor Bets on Rate Increases - WSJ.pdf",
      "score": 0.5803673267364502
    },
    {
      "source": "uploads/1ffebe75_Fed Officials Project Three Interest Rate Rises in 2022 and Accelerate Stimulus Wind-Down - WSJ.pdf",
      "score": 0.5775790214538574
    },
    {
      "source": "uploads/1dd85c28_Stocks End Lower as Bond Yields Hit Two-Year High - WSJ.pdf",
      "score": 0.5645315051078796
    },
    {
      "source": "uploads/e18ee28b_Fed Stakes Out Hawkish
```

Stance on Inflation, Keeps a Wary Eye on Ukraine War \_ Barron's.pdf", "score": 0.5573776960372925}, {"source": "uploads/1554656d\_Fed Set to Raise Rates by 0.75 Point and Debate Size of Future Hikes - WSJ.pdf", "score": 0.552904486656189}, {"source": "uploads/1ffebe75\_Fed Officials Project Three Interest Rate Rises in 2022 and Accelerate Stimulus Wind-Down - WSJ.pdf", "score": 0.5517711639404297}, {"source": "uploads/c9051fb5\_Fed Projects Just One Cut This Year Despite Mild Inflation Report - WSJ.pdf", "score": 0.5470410585403442}, {"source": "uploads/d44afa83\_Fed Raises Interest Rates for First Time Since 2018 - WSJ.pdf", "score": 0.5421205163002014}, {"source": "uploads/9e3184dd\_Booming Job Gains Could Fuel Fed Debate Over Whether More Is Needed to Corral Inflation - WSJ.pdf", "score": 0.54034024477005}, {"source": "uploads/86623b3c\_Fed Pencils In Earlier Interest-Rate Increase - WSJ.pdf", "score": 0.5194504857063293}}](.venv)

## Comments

The rate-hike query returned ten chunks spanning 2021 to 2024, including the December 2021 *WSJ* article on the initial three projected hikes, the March 2022 first-hike article, the 0.75-point hike coverage, and later 2024 commentary on the cutting cycle. The synthesised answer cited specific figures (the 0.25-point March 2022 move, the 1.875% end-of-2022 projection, the 2.75% end-of-2023 projection) traceable to the retrieved sources. This confirms that the pagination fix described is working: the index now spans the full corpus rather than the first 1,000 files.

## Resistance to false premises

```
curl -X POST https://b37trqi2xg.execute-api.us-east-1.amazonaws.com/prod/query -H
"Content-Type: application/json" -d '{"query": "After the rates hike in 2022, the Federal
Reserve took a more accomodative monetary policy to deal with US unemployment", "k":
10}'
```

## Output

```
{"query": "After the rates hike in 2022, the Federal Reserve took a more accomodative
monetary policy to deal with US unemployment", "answer": "Based on the context provided,
this statement is not accurate according to what's described in the documents.\n\nThe
documents indicate that:\n\n1. The Fed implemented aggressive rate hikes in 2022-23 to
combat inflation, not to deal with unemployment.\n\n2. According to one source, there were
concerns expressed that \"the lagged impact of the Fed's aggressive moves to combat
in\ufffdation with big rate hikes in 2022-23 will lead to undesirable weakness in the labor
market.\".\n\n3. The context shows that the Fed began lowering rates later (a September
2024 article mentions a quarter-point rate cut), but this appears to be years after the 2022
rate hikes, not immediately after them.\n\n4. One document notes that the labor market
actually stabilized with low unemployment remaining, which \"removing another rationale for
```

rate cuts.\n\nThe sequence described is: aggressive rate hikes in 2022-23 to fight inflation \u2192 eventual rate cuts later (2024-2025) as inflation cooled and to prevent excessive labor market weakness. There is no indication of an immediate pivot to accommodative policy right after the 2022 rate hikes to deal with unemployment.", "sources": [{"source": "uploads/2f21c7df\_The Fed Unveils Its Everything-Old-Is-New-Again Monetary Policy - Barron's.pdf", "score": 0.5492709875106812}, {"source": "uploads/2f21c7df\_The Fed Unveils Its Everything-Old-Is-New-Again Monetary Policy - Barron's.pdf", "score": 0.4718307852745056}, {"source": "uploads/2f21c7df\_The Fed Unveils Its Everything-Old-Is-New-Again Monetary Policy - Barron's.pdf", "score": 0.4380754232406616}, {"source": "uploads/ff77d1af\_Fed Lowers Rates by Quarter-Point, Signals More Cuts Are Likely - WSJ.pdf", "score": 0.43438735604286194}, {"source": "uploads/23204fa8\_The Fed Has Significantly Improved the Odds of a Soft Landing - WSJ.pdf", "score": 0.4230417311191559}, {"source": "uploads/f4dbbcd9\_Fed\u2019s Aggressive Rate Hikes Are a Game Changer - WSJ.pdf", "score": 0.41935133934020996}, {"source": "uploads/ca4869f5\_The Fed Is Scrambling to Tame Inflation While Avoiding a Recession \_ Barron's.pdf", "score": 0.4099966287612915}, {"source": "uploads/cb54f31c\_The Economy Kevin Warsh Is Inheriting Is Not the One the Next Fed Chair Wanted - WSJ.pdf", "score": 0.4074212610721588}, {"source": "uploads/c4472601\_Is the U.S. Economy In Recession\_ What the Fed\u2014and Everyone Else\u2014Got Wrong. - Barrons.pdf", "score": 0.4061216413974762}, {"source": "uploads/ca4869f5\_The Fed Is Scrambling

## Comments

The second query embedded an incorrect assertion ("the Federal Reserve took a more accommodative monetary policy to deal with US unemployment" after 2022). Rather than echoing the premise, the model opened with "this statement is not accurate" and reconstructed the actual sequence — restrictive stance maintained through 2022–2023, soft-landing commentary in 2024, and the first cut in late 2024 — citing *Barron's* and *WSJ* articles dated accordingly. This is the expected behaviour from a grounded RAG prompt ("If the answer is not in the context, say so"), but it is worth observing that the model also actively contradicts the user when the retrieved context warrants it, rather than only refusing to answer.

## Comparing crypto frenzy with 2006-7 subprime

```
curl -X POST https://b37trqi2xg.execute-api.us-east-1.amazonaws.com/prod/query -H
"Content-Type: application/json" -d '{"query": "I believe cryptocurrencies, SPAC, and meme
stock pose financial systemic risks that are again either ignored or not properly taken into
account such as the subprime that led to the 2008 crisis", "k": 10}'
```

## Output

{"query": "I believe cryptocurrencies, SPAC, and meme stock pose financial systemic risks that are again either ignored or not properly taken into account such as the subprime that led to the 2008 crisis", "answer": "Based on the context provided, there are **mixed views** on whether cryptocurrencies, SPACs, and meme stocks pose systemic financial risks:\n\n**Arguments suggesting systemic risk concerns:**\n1. **European Central Bank warning:** The ECB warned that the crypto market was similar in size to securitized subprime mortgages before the 2008 financial crisis, stating that crypto assets 'will pose a risk to financial stability' if they keep growing and banks increasingly get involved.\n2. **Regulatory concerns:** Lee Reiners from Duke University's Global Financial Markets Center stated: 'The market at this point isn't big enough to trigger a systemic risk event, but these are not static markets. They are continuously evolving and growing. It's time to sound the alarm bells.'\n3. **International warnings:** The FSB (Financial Stability Board) warned that equity and crypto markets pose risks to financial stability, particularly in countries where crypto is taking off.\n**Arguments suggesting limited systemic risk:**\n1. **Limited integration:** Amy Lynch, a former financial regulator, suggested: 'As long as the major financial players are actually not offering their own crypto exchanges and cryptocurrency trading platforms, then we are quote-unquote safe from the systemic' risk.\n2. **Market isolation:** The context notes that 'The latest fiasco hasn't dented broader financial markets. The S&P 500 index is up 3.7% since FTX fell apart.'\n\nThe context does not specifically address SPACs in relation to systemic risk.", "sources": [{"source": "uploads/ec951b9e\_Crypto Took Wall Street on a Wild Ride. Now It\u2019s Ending in Tears. \_ Barron's.pdf", "score": 0.50547856092453}, {"source": "uploads/de01f938\_Weird Finance\_ How to Navigate Cryptos, NFTs, and Other Bubbly Stuff \_ Barron's.pdf", "score": 0.5047844648361206}, {"source": "uploads/de01f938\_Weird Finance\_ How to Navigate Cryptos, NFTs, and Other Bubbly Stuff \_ Barron's.pdf", "score": 0.4270709753036499}, {"source": "uploads/70217f82\_Watchdogs Warn of 'Contagion' as Cryptos and Stocks Surge \_ Barron's.pdf", "score": 0.4142434000968933}, {"source": "uploads/6cd00c6d\_EZB\_ Christine Lagarde warnt vor systemgefu\u00e4hrdenden Cyberattacken.pdf", "score": 0.41128361225128174}, {"source": "uploads/222bad3d\_A Year After It Began, Meme-Stock Mania Is on Life Support \_ Barron's.pdf", "score": 0.40143975615501404}, {"source": "uploads/03304054\_Bitcoin\_ Kryptow\u00e4hrung besch\u00e4ftigt ranghohe Politiker und Banker.pdf", "score": 0.39692920446395874}, {"source": "uploads/ec951b9e\_Crypto Took Wall Street on a Wild Ride. Now It\u2019s Ending in Tears. \_ Barron's.pdf", "score": 0.3960384130477905}, {"source": "uploads/57da4f11\_Where Are Markets Headed\_ Six Pros Take Their Best Guess - WSJ.pdf", "score": 0.39395788311958313}, {"source": "uploads/f1c5bc5a\_Crypto\u2019s Survival Depends on Washington Now \_ Barron's.pdf", "score": 0.39015328884124756}]}(.venv)

## Comments

The answer was structured as "Arguments FOR" and "Arguments AGAINST," quoting the ECB's explicit subprime comparison, a Duke regulator's "sound the alarm bells" remark, and the counter-view that traditional financial players' limited crypto exposure contained the FTX collapse. The model also flagged: *"The documents don't specifically address SPACs in*

*relation to systemic risk beyond mentioning them as speculative assets.*" This kind of explicit acknowledgement of what is *not* in the context is a useful indicator that the system is grounded rather than hallucinating.

**Cross-language retrieval.** The crypto query surfaced two German-language sources (an EZB cyberattack article and a German Bitcoin policy piece) alongside English-language *WSJ* and *Barron's* articles. Titan v2 embeddings are multilingual, and the FAISS retrieval correctly placed semantically related German chunks within the top-10 even though the query was in English. The Claude model then summarised the relevant points in the answer language without any explicit translation step.

## Cost

The FAISS-on-S3 architecture is cheaper than the OpenSearch-based alternative originally considered. Storing the index in S3 costs cents per month rather than the \$70–\$100/month of a managed OpenSearch domain, and no NAT Gateway is required since all Lambdas reach Bedrock and S3 over the AWS network. Total fixed infrastructure cost is about \$5–\$10/month, with Bedrock invocation costs scaling linearly with usage including the Titan embedding calls during indexing, and the Claude calls during querying.

| Component    | OpenSearch Solution     | FAISS Solution       |
|--------------|-------------------------|----------------------|
| Vector Store | ~\$70-100/month         | ~\$0.50/month (S3)   |
| NAT Gateway  | ~\$30-45/month          | \$0 (not needed)     |
| Lambda       | ~\$5/month              | ~\$5/month           |
| Bedrock      | Usage-based             | Usage-based          |
| <b>Total</b> | <b>~\$105-150/month</b> | <b>~\$5-10/month</b> |

## Results and Discussion

The serverless RAG system processes approximately 3,300 financial press articles spanning 2020–2026, producing a FAISS index of 50,727 vectors across 3,281 unique sources. Query latency typically ranges from 3–8 seconds, dominated by the Claude Sonnet 4.5 inference call rather than FAISS retrieval.

## Github

The code is available [here](#).

## Retrieval Quality and Groundedness

Test queries demonstrate semantically relevant retrieval across the full temporal range of the corpus, with cross-language behavior (German sources appearing for English queries) confirming Titan v2's multilingual capabilities. The system exhibits appropriate grounding: when presented with false premises, the model explicitly contradicts the user rather than echoing the assertion, and when information is absent from retrieved context, it acknowledges the gap.

## Comparison with the Local Llama 2 Prototype

The AWS implementation eliminates the constraints that shaped the [earlier prototype](#): 6 GB VRAM limitation, 4-bit quantization, low-dimensional embeddings, document truncation to 1,000 characters, and the two-step summarize-then-answer prompt strategy. The two-step approach has been replaced with single-step retrieved-context prompting, since Claude on Bedrock has no memory constraint analogous to the local system.

## Features Proposed but not implemented

Several enhancements from the Llama 2 prototype document remain unimplemented:

- **Metadata-aware retrieval with date weighting** — the proposed scoring formula  $\text{final\_score} = \alpha \cdot (1-d) + \beta \cdot \text{recency}$  is not implemented; retrieval is purely by embedding similarity.
- **Time-sensitivity filtering** — no automatic detection of temporal phrases like "in recent years."
- **Relevance threshold with fallback** — the system always passes top-k chunks regardless of similarity score.
- **Timestamp disclaimers** — not systematically enforced in responses.
- **Chunk-based retrieval** — partially addressed; the Document Processor chunks PDFs into ~1,000-char segments with 200-char overlap, but within-document pivoting windows are not implemented.

## Toward Agentic AI in AWS

This project serves as a stepping stone toward an agentic AI implementation using AWS Bedrock and other AWS components. The current architecture is a single-turn RAG pipeline; an agentic extension would add query planning, iterative retrieval, multi-step reasoning with memory, and tool integration (live data feeds, calculation engines, citation verification). The cost and latency characteristics of the FAISS-on-S3 architecture—index

cached in Lambda memory, stateless design enabling parallel invocations—make it well suited as a retrieval tool within an agentic loop.

## Limitations

The system lacks incremental index updates (rebuilding the full index takes 2–3 minutes for 3,300 documents), and does not implement confidence scoring or answer verification.